

Task 1 — Web App QA & Debug Report

Application: RealWorld Conduit Demo | URL: <https://demo.realworld.io> | Tester: Candidate | Date: 25 May 2026

1. Executive Summary

The RealWorld Conduit demo (a Medium-clone, Angular frontend + Node/Express API) was evaluated across all major user flows: registration, login, article CRUD, commenting, favouriting, profile management, and logout. Eight issues were identified — three Critical, one High, two Medium, one Medium, and one Low. Five are documented in the bug table below; Issue 3 is analysed in depth in Section 3.

2. Bug Report Table

#	Title / Summary	Steps to Reproduce	Expected vs Actual	Sev.	Suspected Cause
1	JWT Stored in localStorage — XSS Token Theft	1. Register / log in. 2. Open DevTools > Application > Local Storage. 3. Key 'jwtToken' visible in plaintext.	Expected: HttpOnly cookie inaccessible to JS. Actual: Token in localStorage; any injected script can read it with localStorage.getItem().	Critical	Frontend stores JWT via localStorage.setItem() on login success. No HttpOnly / Secure cookie used.
2	No Rate Limiting on Login — Brute-Force Possible	1. Send 50+ POST /api/users/login requests with wrong passwords. 2. All return 422; none are blocked or throttled.	Expected: 429 Too Many Requests or account lockout after N failures. Actual: Unlimited guesses accepted; passwords can be brute-forced.	Critical	No rate-limiting middleware (e.g. express-rate-limit) applied to the login route in the Express API.
3	Stored XSS via Markdown Article Body	1. Log in > New Article. 2. Enter raw HTML script tag in Body field. 3. Publish. 4. Open article as any user.	Expected: Script tag escaped; displayed as literal text. Actual: Script executes in reader's browser, enabling session hijack or defacement.	Critical	marked parser not sanitised; Angular bypassSecurityTrustHtml() disables built-in XSS protection. DOMPurify absent.
4	Double-Click on Publish Creates Duplicate Articles	1. Fill New Article form. 2. Double-click 'Publish Article' quickly. 3. Check profile — two identical articles appear.	Expected: One article; button disabled after first click. Actual: Two POST /api/articles fire before route change; duplicates stored.	High	Submit button not disabled on first click. No optimistic UI lock or loading spinner prevents double submission.
5	Pagination Shows When Articles Fit on One Page	1. Go to a tag / profile with fewer than 10 articles. 2. Observe pagination bar at bottom.	Expected: Pagination hidden when total <= page size. Actual: Page '1' button renders and is clickable; triggers redundant re-fetch with no change.	Medium	Pagination component always renders; Math.ceil(total/limit) = 1 when total < limit. No conditional hide logic.

3. Root-Cause Analysis — Issue #3: Stored XSS via Markdown Body

What is happening?

When a logged-in user publishes an article whose body contains raw HTML tags such as `<script>alert('XSS')</script>`, the script executes in the browser of every subsequent reader. This is a classic **Stored (Persistent) Cross-Site Scripting (XSS)** vulnerability — the payload lives in the database and attacks every future visitor.

Why does it happen?

The Angular frontend renders article content by piping the raw body string through the **marked** Markdown parser, then binding the HTML output directly to the DOM via [innerHTML]. Two compounding failures drive the bug:

1. **marked is not sanitised.** By default, marked passes raw HTML blocks through unchanged. The deprecated 'sanitize' option was never enabled and no external library (DOMPurify) is used to strip dangerous tags.
2. **Angular's DomSanitizer is bypassed.** The component calls `bypassSecurityTrustHtml()` to allow legitimate Markdown formatting (bold, code blocks, links) to render. This call also suppresses Angular's XSS guard, leaving the door open for any malicious tag to execute.

Impact

Compounded by Issue #1 (JWT in localStorage), an attacker can steal any reader's session token with a single document.cookie / localStorage read, silently submit API requests on the victim's behalf, or redirect visitors to phishing pages. Because the payload is persisted server-side, every future reader is affected until the article is deleted.

How to fix it

Server-side (primary fix): Sanitise HTML on ingest in the Express POST /api/articles handler using sanitize-html or DOMPurify (Node build) with a strict allowlist: p, strong, em, code, pre, blockquote, ul, ol, li, a[href]. Reject or strip all other tags before saving to MongoDB.

Client-side (defence-in-depth): Remove `bypassSecurityTrustHtml()`. Pass marked output through `DOMPurify.sanitize()` before binding to [innerHTML]. This protects users even if a stale payload already exists in the database.

Content-Security-Policy (additional layer): Set script-src 'self' in HTTP response headers so that even injected inline scripts are refused by the browser regardless of DOM content.

Verification: After the fix, create a test article with `<script>document.title='pwned'</script>` in the body and confirm the page title does not change and the raw tag is visible as escaped text.

4. Additional Issues Found (Issues 6–8)

Three further issues were identified beyond the five documented in the main table.

Missing ARIA Labels on Icon-Only Buttons (Accessibility / WCAG 2.1 SC 4.1.2) — Severity: Medium

Steps: Tab through UI with a screen reader; focus the heart (favourite) button on an article card.

Expected vs Actual: Expected: Screen reader announces 'Favourite article' or similar descriptive label. Actual: Announces only the numeric count (e.g. '42') — the button purpose is unintelligible to assistive technology users.

Suspected Cause: The button element contains only an icon tag and a count span. No aria-label, title, or visually-hidden text is provided.

Non-Existent Profile Returns HTTP 200 with Empty UI Instead of 404 — Severity: Medium

Steps: Navigate to `///profile/userThatDefinitelyDoesNotExist99999`. Observe blank profile page. Check Network tab.

Expected vs Actual: Expected: 404 status code and a clear 'User not found' message. Actual: HTML shell returns 200; page renders silently empty. Monitoring tools and SEO crawlers misread this as success.

Suspected Cause: SPA always returns 200 for the shell. Angular route swallows the API 404 and renders with no data and no error state.

Tag Field Accepts Arbitrary-Length Input — Breaks Tag Cloud UI — Severity: Low

Steps: 1. Go to New Article. 2. Enter a 500-character string in Tags field and press Enter. 3. Publish. 4. View home page tag cloud.

Expected vs Actual: Expected: Tags longer than ~30 chars rejected with a validation error. Actual: 500-char tag stored; breaks tag cloud layout on the home page.

Suspected Cause: No maxlength attribute on the tag input and no length validation in the Mongoose schema for the tags array items.

5. Testing Methodology

Happy-path testing: Registered, logged in, created/edited/deleted articles and comments, toggled favourites, followed users, logged out.

Boundary / negative input: Blank forms, 500+ char strings, special chars (`<` `>` `'` `"` `;`), and emoji in all text fields.

Security probing: XSS payloads in article body and title; browser storage inspection; unauthenticated API calls; HTTP response header review.

Accessibility audit: Keyboard-only tab navigation; ARIA inspection via axe DevTools browser extension.

API-level testing: Crafted HTTP requests (cURL / Postman) to `api.realworld.io` validating server-side validation, rate limiting, and status codes.
